Лекция 3 Версионирование данных и воспроизводимые вычисления

Курс: Управление процессами машинного обучения MLOps

Продолжительность: 2 академических часа

Связанная работа: ЛР3 «Версионирование данных и воспроизводимые вычисления»

Цель лекции

Объяснить, как связать состояние кода и данных так, чтобы другой участник мог восстановить конкретный эксперимент. После лекции студент должен различать Git commit, DVC metafile, cache и remote, понимать механизм `dvc add`, `push`, `pull` и `checkout`, а также уметь сформулировать проверяемый протокол воспроизводимости.

План занятия

Время	Содержание	Действие преподавателя
0–10 мин	Проблема «тот же код, другие данные»	Разобрать неудачное воспроизведение эксперимента
10–25 мин	Git и границы его применения	Сопоставить код, метаданные и крупные бинарные объекты
25–45 мин	Архитектура DVC	Показать workspace, `.dvc`, cache и remote
45–60 мин	Жизненный цикл версии данных	Выполнить add, commit, push, switch, pull
60–72 мин	Воспроизводимые вычисления	Связать параметры, данные, окружение и результат
72–82 мин	Отказы и диагностика	Разобрать повреждение, недоступный remote и рассинхронизацию
82–90 мин	Инструктаж к ЛР3 и мост к MLflow	Зафиксировать доказательства и индивидуальную защиту

1 Почему одного Git commit недостаточно

Начните с истории: два студента запускают один commit и получают разные метрики. Причины могут быть в разных версиях CSV, порядке строк, границах временных частей, зависимостях или параметрах. Git хорошо версионирует исходный код и небольшие текстовые файлы, но хранение каждой версии крупного набора раздувает репозиторий и усложняет обмен.

Предложите студентам формулу состояния эксперимента:

```
результат = f(код, данные, параметры, окружение, случайность)
```

Если хотя бы один компонент не идентифицирован, фраза «у меня тот же эксперимент» не проверяема. На этой лекции основное внимание уделяется данным, но постоянно возвращайте аудиторию к полной формуле.

2 Что хранит Git, а что хранит DVC

Git должен содержать код, конфигурацию, контракт, небольшие разрешённые примеры, `.dvc`-метафайлы, `dvc.yaml`, `dvc.lock` и документацию. Крупные или часто меняющиеся данные размещаются вне Git.

DVC добавляет слой адресации содержимого. После `dvc add data/file.csv` в рабочем дереве появляется метафайл `data/file.csv.dvc`, а объект помещается в cache. В метафайле находится hash, путь и дополнительные свойства. Git версионирует метафайл; DVC по нему находит нужный объект.

Подчеркните: `.dvc`-файл не содержит сам набор. Если cache пуст и remote недоступен, одного Git clone недостаточно. Поэтому утверждение «всё сохранено в Git» ошибочно.

3 Четыре области DVC

Нарисуйте схему:

```
workspace ↔ cache ↔ remote

↑

Git хранит metafile
```

Разберите каждую область:

- workspace — рабочая копия, которую читает код;
- metafile — версия ссылки на содержимое, отслеживаемая Git;
- cache — локальное содержимое, адресуемое по hash;
- remote — внешнее хранилище для обмена и восстановления.

`dvc add` связывает workspace с cache и обновляет metafile. `dvc push` передаёт объекты из cache в remote. `dvc pull` получает недостающие объекты и восстанавливает workspace. `dvc checkout` приводит workspace к версии, указанной текущими метафайлами, используя доступный cache.

Не формируйте ложное представление, будто DVC автоматически создаёт бизнес-версию набора. Смысл изменения, источник и допустимость использования остаются ответственностью команды.

4 Версия данных как пара Git и DVC

Рассмотрите последовательность v1:

1. Валидатор ЛР2 подтверждает контракт и хронологический порядок.
2. `dvc add` обновляет hash.
3. `dvc push` гарантирует наличие объекта в remote.
4. Git commit фиксирует метафайл, контракт и протокол.

Для v2 процесс повторяется после осмысленного изменения. Получаются два Git commit и два DVC hash. Чтобы восстановить v1, участник переключается на соответствующий commit, получает объект через DVC и снова запускает контракт.

Отдельно обсудите порядок операций. Git commit без `dvc push` может ссылаться на объект, которого нет у коллег. `dvc push` без commit отправляет объект, но не сообщает, какое состояние проекта его использует. Оба действия нужны для совместной воспроизводимости.

5 Cache не является резервной копией

Cache ускоряет работу и устраняет дублирование, но может быть очищен. Remote обеспечивает обмен и восстановление, однако тоже требует политики доступа, резервирования и удаления. Локальный учебный remote удобен для лабораторной, но не моделирует надёжность промышленного объектного хранилища.

Обсудите типичные ошибки:

- cache случайно добавлен в Git;
- remote расположен внутри рабочего дерева;
- в общей конфигурации сохранён персональный абсолютный путь;
- ключ доступа попал в `.dvc/config`;
- объект есть только в cache автора и не отправлен в remote;
- один и тот же путь используется для наборов с разным смыслом без изменения контракта.

6 Контрольные суммы и идентификаторы

В протоколе ЛР3 используются три идентификатора:

Идентификатор	Что фиксирует	Чего не доказывает
Git commit SHA	состояние кода и метаданных	наличие объекта данных в remote
DVC hash	объект, отслеживаемый DVC	законность, качество и предметный смысл
SHA-256 файла	точные байты проверенного CSV	связь с кодом и правилами обработки без дополнительной записи

Объясните, что дублирование здесь полезно: каждый идентификатор отвечает на отдельный вопрос. Validation report соединяет их с версией контракта, периодом и результатом проверок.

7 Воспроизводимые вычисления шире версионирования данных

Версия данных необходима, но недостаточна. Нужно зафиксировать:

- Git commit кода;
- версию данных и контракт;
- границы train, validation и test;
- параметры и random seed;
- версии Python и зависимостей;
- команду запуска;
- полученные метрики и артефакты.

На следующей неделе MLflow станет журналом запусков. Сформулируйте границу ответственности: DVC отвечает за получение нужных данных, MLflow — за регистрацию выполнения и результатов. Ни один инструмент не отменяет Git и автоматические тесты.

8 DVC pipeline и отдельные `.dvc`-файлы

ЛР3 использует отдельный `.dvc`-файл, чтобы сделать механизм прозрачным. В дальнейшем воспроизводимые стадии можно описывать в `dvc.yaml`, а фактические зависимости и выходы — в `dvc.lock`. Команда не должна преждевременно усложнять pipeline: сначала требуется доказать восстановление двух версий исходных данных.

Объясните принцип: стадия пересчитывается, если изменились команда, зависимость или параметр, влияющий на выход. Но DVC не способен определить предметную правильность команды; это обеспечивают контракт, тесты и review.

9 Диагностика отказов

Разберите четыре сценария:

1. `dvc status` показывает изменённый файл — workspace не соответствует метафайлу.
2. `dvc pull` не находит объект — версия не отправлена в remote или выбран неправильный remote.
3. Hash совпадает, но контракт падает — возможно, контракт изменился или ранее была принята плохая версия.
4. Контракт проходит, но метрика не повторяется — проверяются параметры, окружение и случайность.

Главный навык — не «починить любой ценой», а локализовать слой несоответствия. Запрещайте использовать `git reset --hard` как универсальную очистку: команда может потерять незакоммиченный вклад.

10 Командная и индивидуальная работа

Для команды 2–5 человек предложите роли: владелец версии данных, DVC engineer, инженер воспроизводимости, quality reviewer и documentation reviewer. Роли ротируются относительно ЛР2.

На индивидуальной защите преподаватель назначает v1 или v2. Студент самостоятельно переключается на commit, получает данные, запускает валидатор и показывает Git SHA, DVC hash и SHA-256. Затем отвечает, на

каком слое следует искать конкретный отказ.

11 Вопросы аудитории

1. Почему Git LFS и DVC решают не полностью одинаковые задачи?
2. Почему `.dvc`-файл нужно коммитить, а cache — нет?
3. Чем `dvc pull` отличается от `dvc checkout`?
4. Что произойдёт, если автор сделал commit, но не выполнил push?
5. Почему совпавший hash не доказывает качество данных?
6. Какие элементы кроме данных нужны для воспроизведения метрики?
7. Где должны храниться секреты доступа к remote?
8. Почему восстановленную версию нужно снова проверять контрактом?

12 Завершение

Подведите итог: воспроизводимость — это возможность восстановить состояние и проверить результат, а не наличие папки с названием `final`. Git commit и DVC hash образуют адрес состояния, validation report подтверждает допустимость данных, а ЛР4 добавит к этому адресу параметры, метрики и модель в MLflow run.

Чек-лист преподавателя

- Показана разница между workspace, metafile, cache и remote.
- Продемонстрирована связка Git commit и DVC hash.
- Разобраны `add`, `push`, `pull`, `checkout` и `status`.
- Объяснено, почему cache и remote не содержат бизнес-контекст сами по себе.
- Рассмотрены отказные сценарии без разрушительных команд.
- Уточнена индивидуальная демонстрация ЛР3 и связь с ЛР4.

Источники для обновления примеров

- DVC User Guide: https://doc.dvc.org/user-guide
- DVC `.dvc` files: https://doc.dvc.org/user-guide/project-structure/dvc-files